

Assembler, LED, and Text-Based Virtual Pet

Tangina Kader, Amy Showell, Shriya Sistla, Yao Tong

June 16, 2025

1 Assembler Implementation

Our assembler is composed of three main components: the emulate file, where instruction fetch and encode is performed; the modules, which consist of all instruction-encoding functions; and the data structures. The first data structure is the dynamic **symentable**, a dictionary of labels to addresses. The second data structure is the alias table, a dictionary of mnemonics to function pointers where the instruction type is handled.

This assembler utilizes two-pass assembly. The function **first_pass** loops through each line of the input assembly file, storing labels and their corresponding addresses in a symbol table. Next, the function **second_pass** accesses each line of the assembly file again, tokenising each line to identify the instruction name, replace labels with offsets, and extract instruction parameters. The instruction name is then inputted to the alias table's lookup function, which returns a function pointer to the proper encode function. For example, inputting the **add** instruction to lookup table returns a pointer to function **arith** in module **data-processing**. Each of these functions takes the list of tokens, the size of the token list, and an integer pointer to where the encoded instruction is to be stored. The integer pointer's contents are updated based on the instruction type and tokens. Each function also returns a status code indicating encoding success, either **EXIT_SUCCESS** or **EXIT_FAILURE**. Lastly, the encoded instruction is converted to binary and written to the specified output file. Note that in the second pass, any line consisting of only a label, directive, or newline is automatically skipped.

While writing our assembler, we learned to appreciate the importance of fleshing out useful data structures before implementing other functionality. For example, the dynamic symbol table was originally fixed-length, which is inefficient given there could be any number of labels in an assembly file. Also, the table initially stored addresses as integer pointers instead of just integers, which caused issues when looking up addresses using labels. Thus, thorough brainstorming, testing, and review are always necessary before a data structure is deployed. In addition, our group focused on the challenging implementation task we identified last week: writing utility functions early on. Throughout the assembler task, any function that seemed usable in multiple modules – like extracting immediate values and register numbers – were written in a file called **assembly-utils.c**. As a result, many of our tasks were simplified, and we enjoyed more efficient development.

2 Blinking LED Implementation

To implement Part III, we began by building a simple circuit with one red LED, one 220Ω resistor, one wire to ground, and one wire to the output at GPIO pin 3. Then, we made a file, **led.blink.s**,

in which we started to write our assembly code. Our assembly code has five main sections: `setout`, `sethigh`, `setlow`, `wait`, and a section for int directives.

In the first section, `setout`, we select pin 3 as an output by setting FSEL3 to 001 and storing this value in the corresponding GPIO function selection address.

The next section, `sethigh`, sends a high signal to the LED by setting pin 3 (bit 2) to 1 and storing this value in the GPSET0 register. Additionally, we set a flag in register W9 to signal that the current state is high, before branching to the `wait` section of the code.

The third section, `setlow`, of our code sends a low signal to the LED by setting pin 3 (bit 2) to 1 and storing this value in the GPCLR0 register. This time, we clear the flag register to 0 to signal that the current state is low, before branching to the `wait` section of the code.

In the `wait` section of our code, we include a loop to wait for a fixed amount of time before sending a high or low signal. Before the loop, we set register W3 to 0. Within the loop, we add 1 to register W3 until it is greater than or equal to a set wait time of 0x50000. Once it meets this condition, we check the flag register to see if the current state is high or low. We then branch to the next state depending on the outcome of the comparison.

Finally, the last section of our code includes a set of seven int directives that are loaded into registers at the start of the program. The int directives include: the GPIO function select address, the GPSET0 address, the GPCLR0 address, the wait time, the GPIO configuration for function selection, the GP output configuration for sending high/low signals, and the flag for a high state.

We faced many challenges while writing our assembly code. Our attempts to use the `mov` instruction to load an immediate value into a register yielded incorrect results, which confused us greatly. When we finally realised that our emulator and assembler do not support the functionality for immediate operands in logical instructions, we changed the structure of the code to use the `ldr` instruction combined with int directives to store the set and clear flags in the corresponding GPIO output registers. When using a `mov` instruction was necessary, we changed both operands to be from registers rather than immediate values.

Another challenge we faced while trying to diagnose the previous issue was understanding the code we had written. Since our assembler does not support comments, descriptive variable names and labels were essential to code clarity. However, when we began writing the code, our variable names were extremely generic, causing many delays in debugging from having to constantly remind ourselves what each register and int directive was used for. Therefore, we altered our code to have clearly labelled sections, as well as comprehensible int directive names.

3 Text-Based Virtual Pet

A “virtual pet” is a digital toy that often requires care and attention. The most popular example is Bandai Namco’s Tamagotchi, a physical toy with a few interaction buttons and a digital screen where the “pet” is displayed. Similar to a Tamagotchi, our text-based virtual pet extension requires players to feed, clean, and play with their pet to keep their pet happy and alive. However, our implementation also includes an “interaction limit” to prevent users from becoming addicted or dependent on their virtual pets. Hence, our virtual pet extension can be used as a fun way to help children and teens practice responsibility and balance; although users should check on and care for their virtual pet every once in a while, they are not allowed to spend the entire day focusing only on their pet. This concept can then be generalized to any aspect of life, including studies, self-care, hobbies, and more.

3.1 Virtual Pet Design

3.2 Challenges

3.3 Testing

4 Group Reflection

5 Individual Reflections

5.1 Tangina Kader

5.2 Amy Showell

5.3 Shriya Sistla

5.4 Yao Tong